

MediSeba

Intelligent Healthcare & Prescription System

Your Health, Your Care, Anywhere

Project documentation for judges and organizers

At a glance	
What it is	A full-stack, multi-role web platform (patient, doctor, administrator) for appointment booking, digital prescriptions, payments, AI assistance and video consultation.
Problem	Long waiting times, opaque appointment queues, paper prescriptions and uneven access to specialists in Bangladesh's healthcare system.
Tech stack	PHP 8.1 REST API, MySQL 8.0, HTML5/CSS3/JavaScript, Google Gemini API, Jitsi Meet (WebRTC).
Scale	16 database tables, 2 stored procedures, 2 audit triggers, 3 analytical views, 52 API endpoints.
Source code	https://github.com/mohammademon10/MEDISEBA

1. Problem and Motivation

Bangladesh faces persistent challenges in healthcare delivery: unequal distribution of medical professionals, inefficient appointment management at public hospitals, and no structured digital prescription records. Patients in urban and semi-urban areas endure long waits, lack appointment transparency and struggle to reach specialists, while physicians have few tools to manage schedules, track patient histories or issue structured prescriptions digitally.

MediSeba (meaning 'Medical Service' in Bengali) addresses these gaps. It connects patients with verified doctors, automates scheduling through a token-based queue, produces structured digital prescriptions with PDF export, records payments, and uses Google Gemini AI to help patients and physicians. Growing internet and smartphone adoption in Bangladesh makes a web-based solution increasingly viable, and the platform runs on ordinary Apache-based shared hosting, with no special infrastructure.

1.1 Objectives

- Design a normalized relational schema for multi-role users, scheduling, prescriptions and financial records.
- Build a secure RESTful PHP API with JWT authentication, role-based access control, and protection against SQL injection and CSRF.
- Provide an intuitive interface for three roles: patient, doctor and administrator.
- Assign appointment tokens atomically, sequentially per doctor per day, using MySQL stored procedures.
- Integrate Google Gemini for symptom summarization, medical-term explanation and clinical summary drafting, all server-side.
- Generate prescription and payment-receipt PDFs with a zero-dependency PDF engine.
- Offer real-time doctor-patient video consultation through Jitsi Meet.

1.2 Gap analysis

Gap in existing platforms	How MediSeba responds
No AI assistance for symptom parsing or clinical summaries	Gemini is called server-side, so the API key never reaches the browser.
Appointments booked, but no structured, downloadable prescriptions	Full prescription lifecycle: creation, medicine list, PDF, AI term explanation.
Simple CRUD with little database engineering	Stored procedures for atomic operations, triggers for audit logging, SQL views for dashboards.
No video consultation	Jitsi Meet room URL generated automatically when an appointment is confirmed.
Weak security	Password hashing, HMAC-SHA256 JWT, CSRF protection, PDO prepared statements, rate limiting.

2. System Architecture

MediSeba follows a three-tier pattern: a presentation tier (HTML/CSS/JS pages calling the API through the Fetch API), a logic tier (PHP RESTful API) and a data tier (MySQL). The backend is split into a router (index.php), middleware (JWT validation and role enforcement), controllers (business logic per domain) and models (a PDO-based base class). Configuration lives in a .env file, in line with twelve-factor practice.

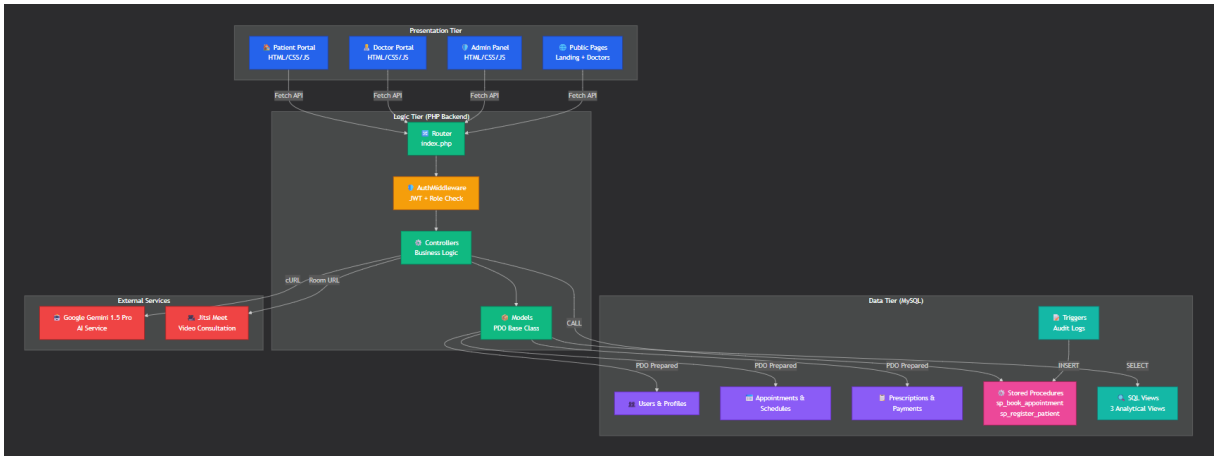


Figure 1: System architecture (Frontend → PHP API → MySQL)

2.1 Appointment booking workflow

A patient browses the doctor directory, selects a doctor, views available slots, picks a date and submits a booking. The backend calls the **sp_book_appointment** stored procedure, which inside one transaction locks the schedule row, counts existing bookings, rejects the request if the slot is full, and otherwise assigns the next sequential token. This removes race conditions under concurrent bookings. When the doctor confirms, a unique Jitsi room URL is generated and a Join Call button appears in both portals.



Figure 2: Appointment booking flow

2.2 AI integration

Three server-side endpoints relay prompts to the Gemini REST API through a GeminiService class: symptom summarization (patient), plain-language medical term explanation (patient), and clinical summary generation (doctor). If the key is missing or the upstream service fails, the system returns a friendly error and the core booking and prescription workflows are unaffected.

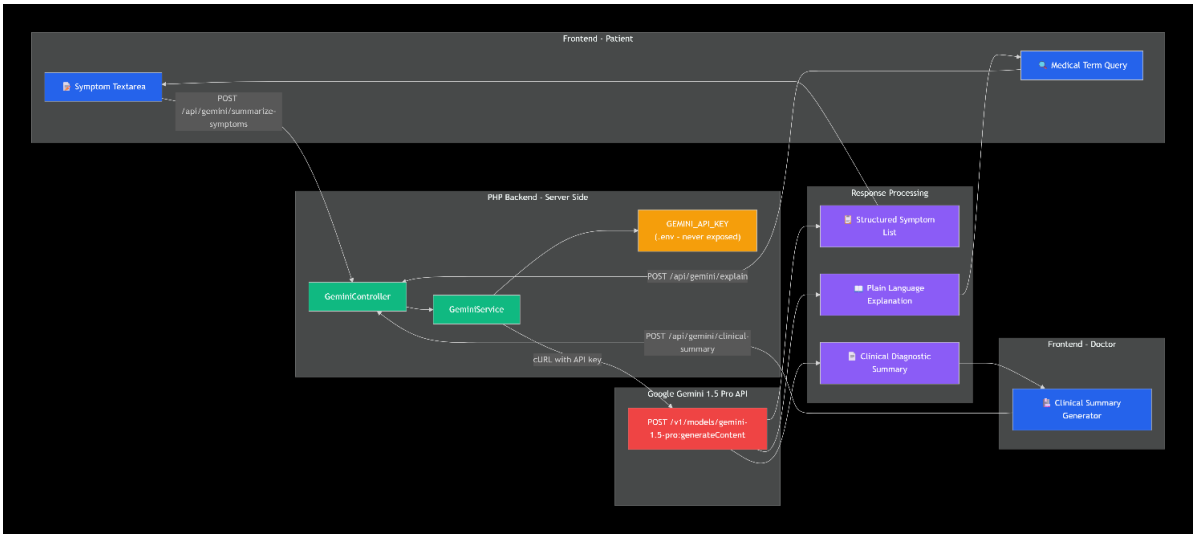


Figure 3: AI integration architecture

3. Database Design

The database has 16 tables with full normalization, referential integrity and indexing on foreign keys, status and frequently filtered columns.

Table	Purpose
users	Root authentication entity: email, password hash, role (patient/doctor/admin), status.
doctor_profiles, patient_profiles	One-to-one role profiles: doctor specialty, qualification, fee and clinic; patient blood group, allergies, chronic conditions, emergency contact.
doctor_schedules	Weekly availability by day, start/end time, slot duration and maximum patients.
appointments	Central transactional table: patient, doctor, schedule, date, token number, status, video room URL.
prescriptions	Diagnosis, symptoms, medicine list (JSON), dosage instructions, advice, follow-up date.
payments	Amount, method, gateway transaction ID and status per appointment.
activity_logs, appointment_status_history	Trigger-populated audit trail with old/new values, acting user and IP address.

SQL view	Purpose
vw_appointment_details	Appointments joined with patient, doctor and schedule data for dashboards.
vw_financial_reports	Payments joined with appointment, doctor and patient data for analytics.
vw_user_master_profile	Unified user and profile view resolving patient versus doctor.

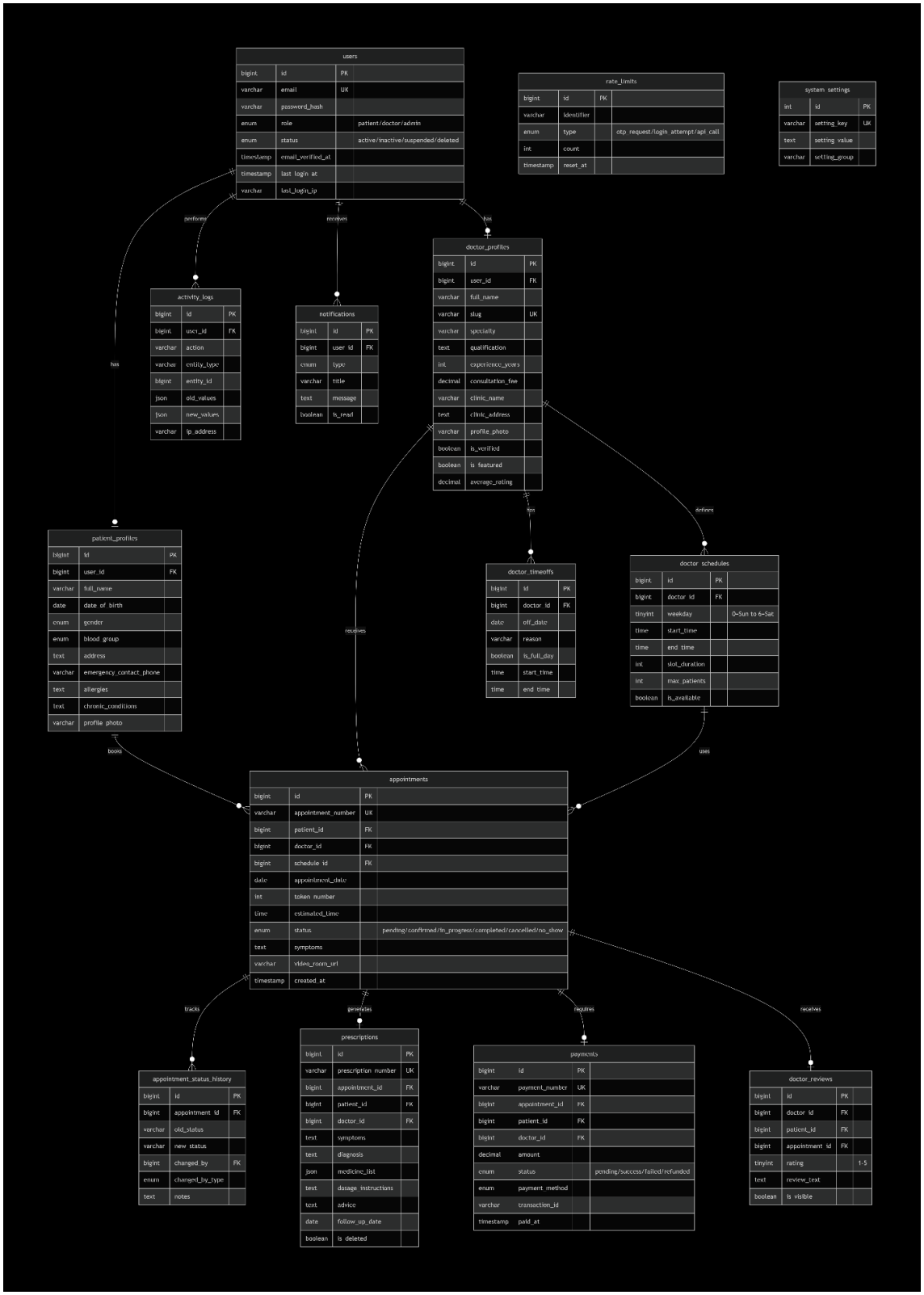


Figure 4: Entity-relationship diagram

4. Implementation Highlights

Authentication and access control

Patient registration calls the **sp_register_patient** stored procedure, creating the user and profile records in one transaction so partial registrations are impossible. JWTs (HMAC-SHA256) carry user ID, email and role; middleware validates the token on every protected request and enforces patient, doctor and admin tiers. Login is rate-limited through a MySQL-backed limiter: after five consecutive failures the endpoint returns HTTP 429 with a retry interval.

Appointments and video consultation

Appointment status follows a state machine: pending → confirmed → in_progress → completed (or cancelled / no_show). Every transition is logged by a database trigger. Confirmation generates a cryptographically random Jitsi room URL, which opens in an embedded modal with keyboard and backdrop dismissal; closing it clears the iframe to end the media session.

Prescriptions and PDF generation

Doctors create structured prescriptions for their own appointments only. A custom pure-PHP class implements the PDF 1.4 specification (cross-reference table, page tree, content streams, built-in fonts) to produce prescription and receipt PDFs without any external library. AI explanations are shown inline and never stored, so the official record stays unaltered.

Administration

Administrators see aggregate statistics, verify or reject doctor registrations, manage user status, cancel appointments, browse prescriptions, access the financial ledger with receipts, and edit system settings. Every action is written to the audit log with the admin's ID and IP address.

Security and performance

- JWT authentication with role-based access control; HTTP 403 for cross-role access.
- PDO prepared statements on every query; CSRF protection; rate limiting on sensitive endpoints.
- Stored procedures reduce round trips; indexes and SQL views keep dashboard queries efficient.
- Server-side AI calls keep the API key out of the browser.

Module	Endpoint	Method	Access
Authentication	/api/auth/login, /api/auth/register	POST	Public
Doctors	/api/doctors	GET	Public
Appointments	/api/appointments	POST	Patient
Appointments	/api/appointments/{id}/status	PATCH	Authenticated
Appointments	/api/appointments/{id}/videoroom	GET	Authenticated
Prescriptions	/api/prescriptions	POST	Doctor
Prescriptions	/api/prescriptions/{id}/pdf	GET	Authenticated
AI	/api/gemini/explain, /summarize-symptoms	POST	Patient
AI	/api/gemini/clinical-summary	POST	Doctor
Admin	/api/admin/doctors/{id}/verify	PATCH	Admin

Table: representative API endpoints (52 in total).

5. User Interface

The interface is a multi-page application with three portals (patient, doctor, admin) sharing one CSS design system: Inter typeface, Font Awesome icons, a primary blue, and status colors (green confirmed, amber pending, red cancelled). Authenticated pages use a sidebar; public pages use a hero layout. A central api.js module handles JWT attachment, errors and toast notifications, and a dark/light mode toggle is available.

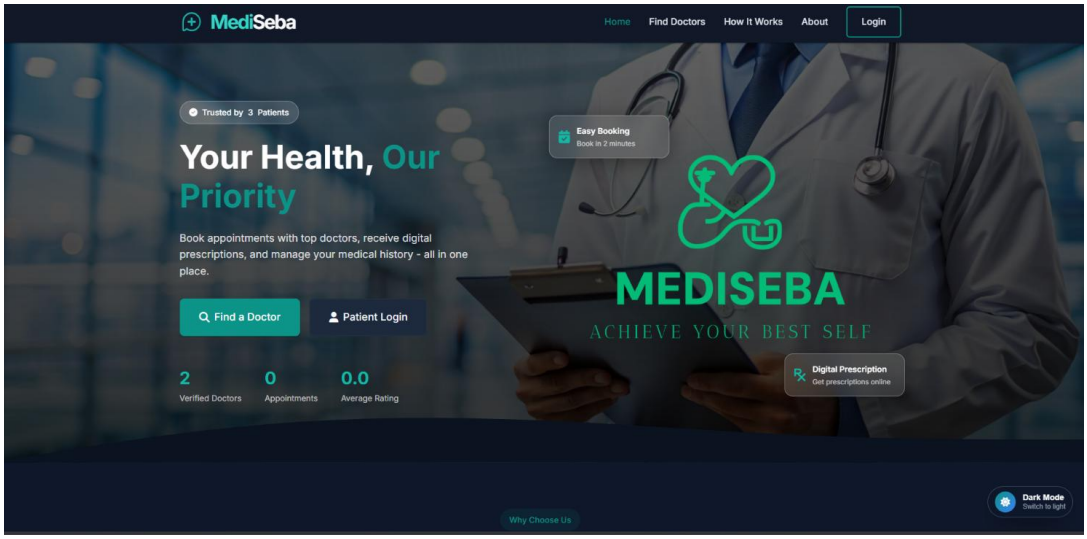


Figure 5: Home page

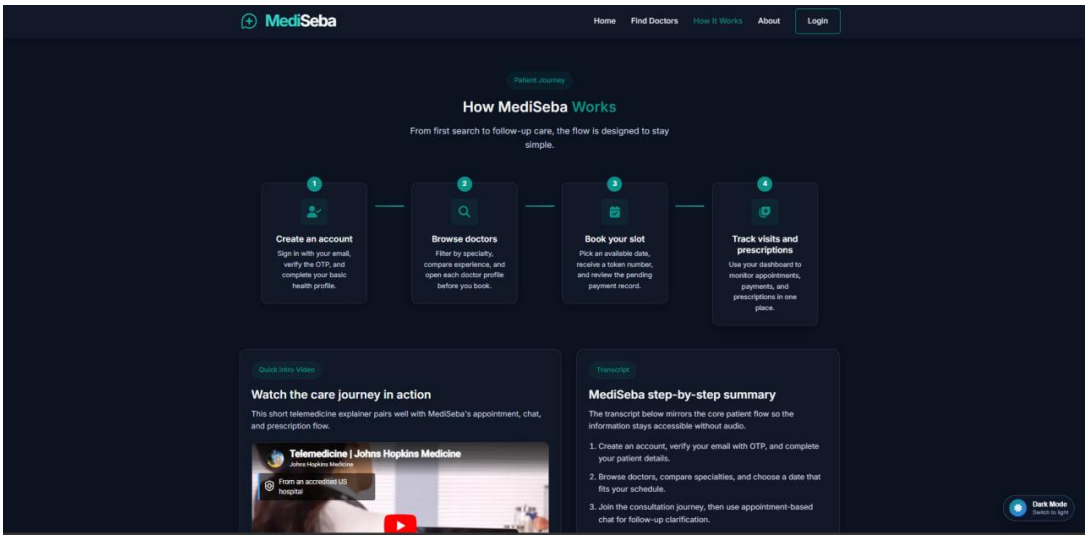


Figure 6: How MediSeba works

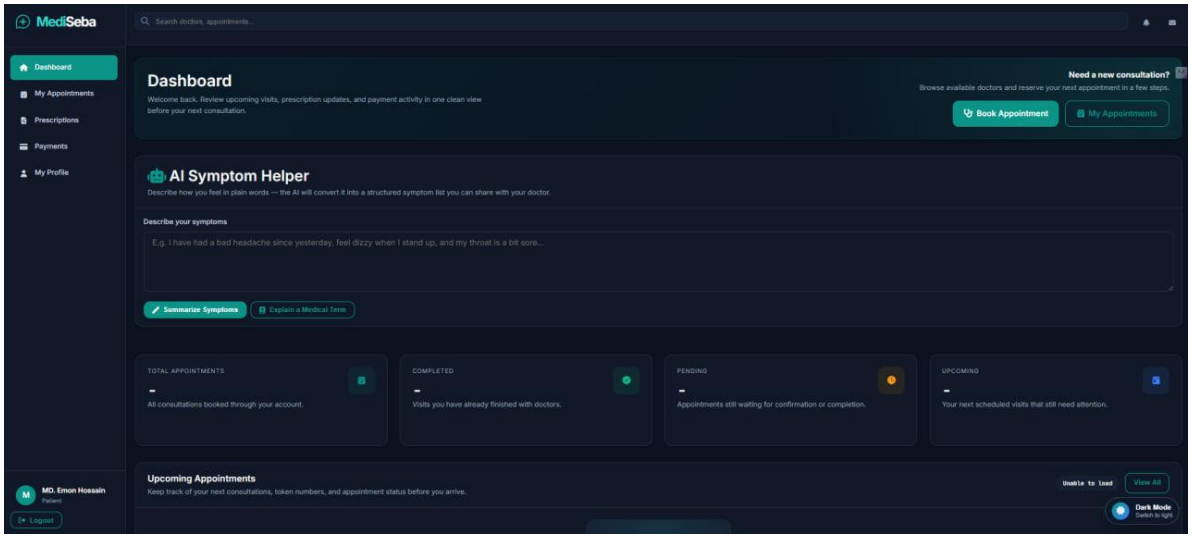


Figure 7: Patient dashboard with AI Symptom Helper

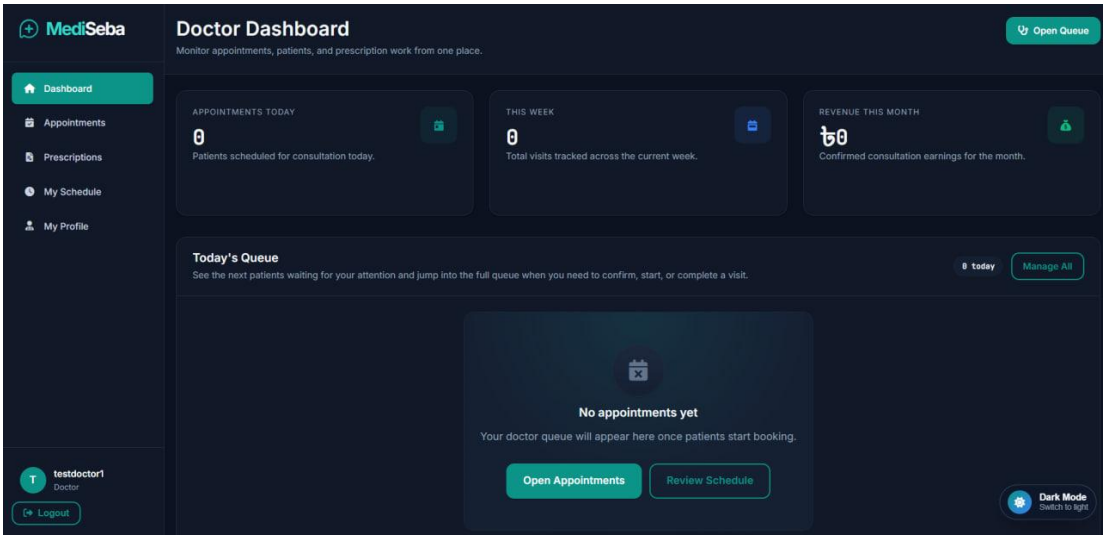


Figure 8: Doctor dashboard and appointment queue

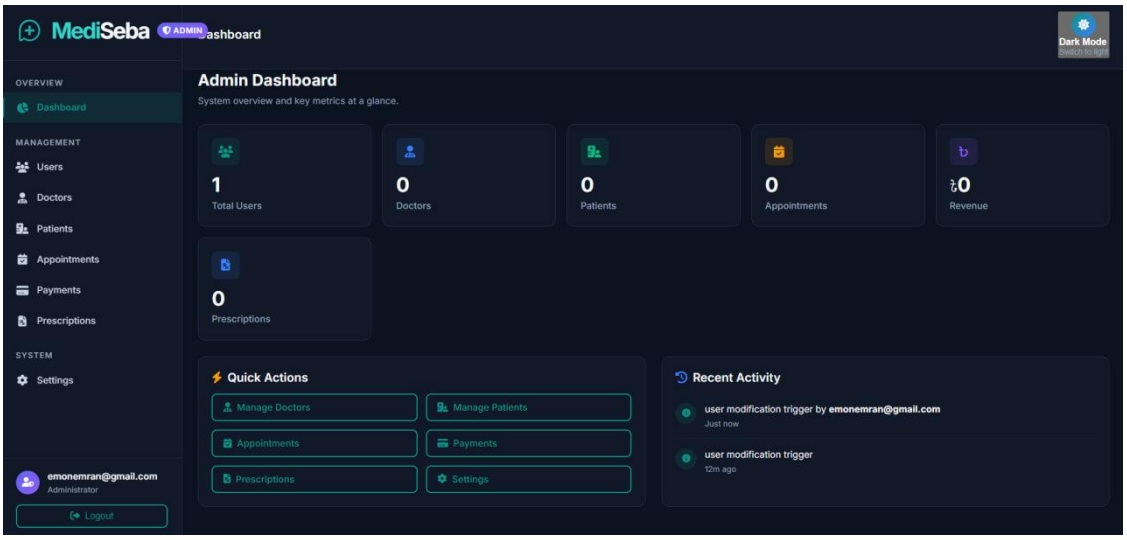


Figure 9: Admin control panel

6. Results

- Concurrent bookings produce no duplicate tokens, verified by manual testing of sp_book_appointment under simultaneous requests.
- JWT middleware returns HTTP 403 when a patient calls doctor-only endpoints and vice versa.
- All three Gemini workflows return structured symptom lists, plain-language explanations and clinical summary drafts when a valid key is configured.
- Generated PDFs open correctly in Adobe Acrobat, Chrome's viewer and Foxit Reader.
- Jitsi video calls were established through the embedded iframe on both doctor and patient portals.
- The admin verification workflow toggles the verified flag and writes the action to the audit log.

7. Impact, Ethics and Sustainability

Impact on people

Patients can find specialists beyond their immediate area, see transparent token positions, keep digital prescription history and understand medical terms through AI explanations, which helps people with limited health literacy in particular. Physicians gain a structured digital workflow, a clear daily queue and AI-assisted drafting that reduces documentation burden. Video consultation extends specialist reach to remote locations and reduces costly travel for follow-ups.

Society and environment

By letting semi-urban and rural patients with internet access reach verified specialists, MediSeba targets the concentration of qualified physicians in cities. Digital prescriptions and records reduce paper use, and remote consultations cut travel-related emissions. Deployment on shared hosting keeps energy use low.

Ethical design

- **Data privacy:** health data is reachable only by authenticated users with the right role; no patient data is exposed on public endpoints.
- **AI accountability:** AI output is labelled as a drafting aid, not a diagnosis, so the physician stays accountable.
- **Verification gate:** only doctors approved by an administrator can practice on the platform.
- **Audit logging:** every status change and admin action is recorded for accountability.
- **Abuse protection:** login rate limiting defends against credential stuffing.

Sustainability

Standard, widely supported technologies (PHP, MySQL, Apache) avoid vendor lock-in and have a large maintainer pool. A modular controller-model structure lets components be replaced independently, and .env configuration allows migration between hosts without code changes. The code is released under the MIT license.

Cost

Resource	Cost
Development environment (XAMPP)	Free, open-source
Shared Apache hosting (production)	About $3–5 per month
Google Gemini API	Free tier
Jitsi Meet video	Free, open-source
Font Awesome and Google Fonts	Free (CDN)
Domain name (optional)	About $10 per year

8. Limitations and Future Work

Current limitations

- No WebSocket or SSE notifications yet; users refresh to see status changes.
- No outbound email (confirmations, reminders); the OTP and EmailJS pieces are only partly configured.
- No live payment gateway; payments are recorded as completed on method selection to simulate the gateway.
- Mobile layouts are not fully optimized; some admin tables scroll horizontally on narrow screens.
- Testing is manual; there is no automated PHPUnit suite yet.
- Identical AI prompts are not cached, so repeated requests make fresh API calls.

Planned improvements

- Progressive Web App with offline support and push notifications.
- Real-time notifications with WebSockets (Ratchet or Soketi).
- Live payments through SSLCommerz or bKash.
- PHPUnit unit and integration tests as a regression safety net.
- Docker Compose packaging for reproducible deployment.
- Full Bengali localization.
- Medicine database with autocomplete, dosage suggestions and interaction checking.
- Expanded AI features: drug-interaction checker, weight- and age-based dosage advisor, follow-up reminder drafting.

Source code

Repository: https://github.com/mohammademon10/MEDISEBA

References

- Google, Gemini API documentation: ai.google.dev/gemini-api/docs
- PHP Manual 8.1: php.net/manual/en; MySQL 8.0 Reference Manual: dev.mysql.com/doc/refman/8.0/en
- IETF RFC 7519, JSON Web Token (JWT); Jitsi Meet Developer Guide: jitsi.github.io/handbook/docs/dev-guide
- OWASP Top Ten Web Application Security Risks (2021): owasp.org/www-project-top-ten
- Fielding, R. T. (2000), Architectural Styles and the Design of Network-based Software Architectures; Elmasri and Navathe, Fundamentals of Database Systems, 7th ed.